

Supplementary Material

Ambient WiFi as a Radar Replacement for Automotive SLAM: A Physics-Based Feasibility Study

Mulham Fetna (ORCID 0009-0006-4432-798X)

This supplement provides the artifact/reproducibility details and the full dataset datasheet that support the main manuscript but exceed its page limit. It duplicates no results; every number and figure in the paper is regenerated by the commands below.

S1. Artifact availability and licensing

All code, scene/run configurations, result artifacts, and the dataset are openly available and permanently archived.

- **Repository:** <https://github.com/mulhamfetna/wifi-radar-slam>
- **Zenodo concept DOI** (all versions): [10.5281/zenodo.21247288](https://doi.org/10.5281/zenodo.21247288)
- **License:** AGPL-3.0-or-later. **Environment:** Python ≥ 3.11 ; Sionna RT 2.0 (CPU/LLVM or CUDA) for simulation; scikit-learn for the discriminator; CSISKit for the real-CSI proof-of-concept.
- **Full quality:** prepend `WRS_NUM_SAMPLES=1000000` to any run.

S2. Reproducibility recipe

Each element of the paper is produced by a single command from a released config:

Paper element	Command (from the repository root)
Localization (Phase A)	<code>python experiments/run_phase_a.py configs/nominal.yaml</code>
Operating envelope (Table I)	<code>python experiments/run_phase_b.py</code>
Oracle map, controlled wall	<code>run_phase_a.py configs/controlled_oracle.yaml</code>
Oracle map, reflective canyon	<code>run_phase_a.py configs/street_metal_oracle.yaml</code>
Realistic map (joint MUSIC)	<code>run_phase_a.py configs/controlled_music_joint.yaml</code>
60 GHz test (Table, Sec. VII)	<code>run_phase_a.py configs/controlled_music_60ghz.yaml and ...60ghz_16ant.yaml</code>
Scene render (Fig. 2)	<code>python experiments/render_scene.py</code>
Dataset (WiFiSLAM-Sim)	<code>python experiments/make_dataset.py configs/nominal.yaml data/wifislam_sim.npz</code>
Learned discriminator	<code>python experiments/train_discriminator.py data/wifislam_sim.npz</code>
Real-CSI proof-of-concept	<code>sh experiments/fetch_real_csi.sh</code> then <code>python experiments/run_real_csi.py</code>

S3. Wi-FiSLAM-Sim dataset datasheet

A ray-traced outdoor/vehicular Wi-Fi-CSI dataset (no public outdoor/vehicular Wi-Fi-CSI dataset otherwise exists). *Simulated* (Sionna RT), not hardware; intended for algorithm development and as the labelled substrate for path discrimination. Generated by `make_dataset.py`; loaded with `wifi_radar_slam.dataset.CsiDataset.load`.

key	shape	description
<code>csi</code>	$(n_f, n_{ap}, n_{tx}, n_{sc})$	complex64 noisy channel frequency response
<code>poses</code>	$(n_f, 3)$	ground-truth vehicle x, y, yaw (m, m, rad)
<code>ap_positions</code>	$(n_{ap}, 3)$	access-point coordinates (m)
<code>gt_map</code>	$(M, 2)$	ground-truth facade footprints (m)
<code>paths</code>	$(P, 9)$	flat oracle path table (below)
<code>meta</code>	JSON	carrier/bandwidth/subcarriers/antennas/scene/SNR/units

paths columns: `[frame, ap, delay_s, phi_r, theta_r, n_bounce, first_interaction_type, object_id, is_floor]`. The last three are the ground-truth *labels* for learned LOS/reflection/floor discrimination (`first_interaction_type`: 0 none, 1 specular, 2 diffuse, 4 refraction, 8 diffraction). **Tasks:** localization (`csi`→`poses`), mapping (reflectors vs `gt_map`), path discrimination (features from `paths`). **Limitations:** simulated (AWGN only); one straight trajectory per record; sub-7 GHz unless generated from a 60 GHz config.

S4. Real commodity-CSI provenance

The front-end proof-of-concept uses real captures shipped with CSISKit (<https://github.com/Gi-z/CSISKit>, under its licence): Intel 5300 (`log.all_csi.6.7.6.dat`; 30 subcarriers, 3 antennas) and Broadcom nexmon 802.11ac (`example_4358.pcap`). Fetch them with `experiments/fetch_real_csi.sh`. These are indoor/static captures with no ground-truth trajectory; they validate that the sensing front-end runs on measured commodity CSI, not full-system SLAM.